
Lab - Week 8 (Solutions)

Classical Game Theory (Nash Equilibrium)

1. Dominated Strategies

Tasks

3.1 Define formally what a dominated strategy is.

Answer 3.1: For Player 1, a strategy s is dominated by another strategy t if for all possible choices of actions by Player 2, the payoff for t is at least as good as (greater or equal to) the payoff of s and better for at least one choice.

3.2 Explain why a dominated strategy is an important concept.

Answer 3.2: A rational player would never play a dominated strategy

3.3 Find out what the difference is between strictly dominated and weakly dominated. Which of the dominated strategies from 3.3 are weakly dominated, which strictly?

Answer 3.3 : What we have discussed so far are weakly dominated strategies. A strictly dominated strategy delivers a strictly inferior payoff (less than) for all possible actions of the other player.

3.4 We have defined the game *Morra* in the lecture. Which strategies are dominated in *Morra*? For this, recall that the payoff matrix for (our version of *Morra*) is:

$$P = \left(\begin{array}{c|c|c|c} 0 & 2 & -3 & 0 \\ \hline -2 & 0 & 0 & 3 \\ \hline 3 & 0 & 0 & -4 \\ \hline 0 & -3 & 4 & 0 \end{array} \right);$$

Answer 3.4 : *Morra* does not have any dominated strategies (show by elimination of candidates)

3.5 For each of the bi-matrix games below, name the dominated strategies. Which of the dominated strategies are weakly dominated, which strictly?

$$A = \left(\begin{array}{c|c} 2 & 1 \\ \hline 1 & 1 \end{array} \right); \quad B = \left(\begin{array}{c|c} 1 & 1 \\ \hline 1 & 3 \end{array} \right);$$

$$A = \left(\begin{array}{c|c|c|c} 2 & 1 & 3 & 17 \\ \hline 27 & 3 & 1 & 1 \\ \hline 4 & 6 & 7 & 18 \end{array} \right); \quad B = \left(\begin{array}{c|c|c|c} 11 & 9 & 10 & 22 \\ \hline 0 & 1 & 1 & 0 \\ \hline 2 & 10 & 12 & 0 \end{array} \right)$$

$$A = \left(\begin{array}{c|c|c} 3 & 3 & 2 \\ \hline 2 & 1 & 3 \end{array} \right); \quad B = \left(\begin{array}{c|c|c} 2 & 1 & 3 \\ \hline 2 & 3 & 2 \end{array} \right);$$

Answer 3.5:

First example: Strategy 2 for Player A, Strategy 1 for Player B

Second example: Strategy 1 for Player A (by 3), Strategy 2 for Player B (by 3)

Third example: none for Player A, Strategy 1 for Player B (by 3)

Only Strategy 1 for Player A (by 3) in the second example is strongly dominated (by 3).

Answer 3.6:

See separately supplied Jupyter notebook.

2. Nash Equilibrium

Nash Equilibrium in Pure Strategies

A pure Nash Equilibrium contains only one strategy of each player.

Tasks

4.1 Does the game in bi-matrix notation below have a pure Nash equilibrium? Give the equilibrium strategy profile or show why there is no pure Nash equilibrium.

$$\text{In[*]}:= \mathbf{A} = \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}; \mathbf{B} = \begin{pmatrix} 4 & 2 \\ 3 & 6 \end{pmatrix};$$

Answer 2.1: no pure Nash. For Player A: on the cells with payoff 1, Player A would unilaterally move to the other strategy. However, neither of these other outcomes is a candidate for Nash, since Player B would unilaterally switch strategy.

4.2 Does the game from 4.1 have a mixed Nash equilibrium? Give reasons.

Answer 2.2: Yes (without further analysis): Every NF game with finitely many strategies has a Nash equilibrium. To determine mixed Nash follow the procedure from the seminar to equalise the expected payoffs for each player: A-> (3/5, 2/5), B->(1/2, 1/2)

Here it is, step by step:

A plays Strategy 1 with probability p , Strategy 2 with $(1-p)$, so A's strategy vector is

$$\text{In[*]}:= \mathbf{SA} = \{p, 1 - p\};$$

The expected payoff for B given SA is

$$\begin{aligned} \text{In[*]}:= & (\pi\mathbf{B} = \mathbf{SA} \cdot \mathbf{B}) // \text{MatrixForm} \\ \text{Out[*]}//\text{MatrixForm}= & \begin{pmatrix} 3(1-p) + 4p \\ 6(1-p) + 2p \end{pmatrix} \end{aligned}$$

A wants to make B indifferent to their (B's) strategy choice. Set the payoff for both of B's pure strategies equal to find the right p for A to play:

$$\begin{aligned} \text{In[*]}:= & \text{solA} = \text{Solve}[\pi\mathbf{B}[[1]] == \pi\mathbf{B}[[2]], p] // \text{First} \\ \text{Out[*]}:= & \left\{p \rightarrow \frac{3}{5}\right\} \end{aligned}$$

So the mixed strategy for A to achieve their goal is

```
In[*]:= SA /. solA
```

```
Out[*]:=
```

$$\left\{\frac{3}{5}, \frac{2}{5}\right\}$$

This therefore is A's mixed Nash strategy.

Equivalently, for B we have

```
In[*]:= SB = {q, 1 - q};
```

With the expected payoff for A for each strategy choice is

```
In[*]:= (πA = A.SB) // MatrixForm
```

```
Out[*]//MatrixForm=
```

$$\begin{pmatrix} 2(1-q) + q \\ 1 + q \end{pmatrix}$$

So B forces A into indifference if they choose

```
In[*]:= solB = Solve[πA[[1]] == πA[[2]], q] // First
```

```
Out[*]:=
```

$$\left\{q \rightarrow \frac{1}{2}\right\}$$

Thus, B's mixed strategy in Nash is

```
In[*]:= SB /. solB
```

```
Out[*]:=
```

$$\left\{\frac{1}{2}, \frac{1}{2}\right\}$$

4.3 (optional) Define a two-player zero sum game that has a pure Nash equilibrium.

Answer 2.3

For example:

```
In[*]:= A =  $\begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}$ ;
```

```
TableForm[A, TableHeadings → {"A", "B"}, {"A", "B"}]
```

```
Out[*]//TableForm=
```

	A	B
A	1	2
B	0	1

(A/A) is a pure Nash equilibrium.

3. Finding Nash Equilibria by Linear Programming

Some games have an equilibrium only in mixed strategies. In general, this is computationally hard to find, but it can be easily done for 2-person zero sum games. Below you need to find such equilibria using linear programming. As usual, you can do this either in Python or in Mathematica (or, of course, on a piece of paper, but I don't recommend that!).

Finally solving *Morra*

We will now finally solve *Morra*. Recall that the payoff matrix for (our version of *Morra*) is:

$$\text{In}[*]:= \mathbf{P} = \left(\begin{array}{c|c|c|c} 0 & 2 & -3 & 0 \\ -2 & 0 & 0 & 3 \\ 3 & 0 & 0 & -4 \\ 0 & -3 & 4 & 0 \end{array} \right);$$

Tasks

Task 3.1 Use the linear programming support of your preferred system (Python, Mathematica, ...) to determine the mixed Nash equilibrium for *Morra* exactly (or up to numerical precision in Python). As normally we will want a minimisation problem, the most direct way is to write down the system for Player B.

Answer 3.1 (Version 1) : For Python see separate notebook

In Mathematica, we can simply write the problem for Player B down as is .

```

In[*]:= LinearOptimization[
  z,
  {z - 2 y2 + 3 y3 ≥ 0,
   z + 2 y1 - 3 y4 ≥ 0,
   z - 3 y1 + 4 y4 ≥ 0,
   z + 3 y2 - 4 y3 ≥ 0,
   y1 + y2 + y3 + y4 == 1,
   0 ≤ y1 ≤ 1, 0 ≤ y2 ≤ 1, 0 ≤ y3 ≤ 1, 0 ≤ y4 ≤ 1
  },
  {y1, y2, y3, y4, z}
]

Out[*]= {y1 → 0, y2 → 4/7, y3 → 3/7, y4 → 0, z → 0}

```

Note that Morra is a degenerate problem. The support of the mixed strategy contains only y_2 and y_3 . Since these have zero payout, the players can use any choice of probabilities as long as they only play a strategy with this support.

Answer 3.1 (Version 2) :

The above works perfectly by itself. But we can also write this down in Matrix Notation.

The general form the solver demands is: "Minimize $F.Y$ subject to $A.Y+B \geq 0$ and $A1.Y+B1=0$ "

```

In[*]:= F = {0, 0, 0, 0, 1}; (* F.Y is the objective function for Y={y1,y2,y3,y4,z} *)

```

Now the inequality constraints (the last column is z):

$$\text{In[*]:= } \mathbf{A} = \begin{pmatrix} 0 & -2 & 3 & 0 & 1 \\ 2 & 0 & 0 & -3 & 1 \\ -3 & 0 & 0 & 4 & 1 \\ 0 & 3 & -4 & 0 & 1 \\ -1 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix};$$

In[*]:= $\mathbf{B} = \{0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0\};$

The top third of the matrix A captures the actual payoff function, the second ($y_i \leq 1$) and third part ($y_i \geq 0$) simply limit all x to the range $0 \leq x \leq 1$.

Now for the equality constraint. We only need to ensure that all strategy probabilities sum up to 1.

In[*]:= $\mathbf{A1} = \{\{1, 1, 1, 1, 0\}\};$

$\mathbf{B1} = \{-1\};$

Now we are ready to solve "Minimize F.Y subject to $\mathbf{A} \cdot \mathbf{Y} + \mathbf{B} \geq 0$ and $\mathbf{A1} \cdot \mathbf{Y} + \mathbf{B1} = 0$ ".

In[*]:= `LinearOptimization[F, {A, B}, {A1, B1}]`

Out[*]:= $\left\{0, \frac{4}{7}, \frac{3}{7}, 0, 0\right\}$

If you want to check what we have written above, it's easiest to define the vector of decision variables

In[*]:= $\mathbf{Y} = \{y1, y2, y3, y4, z\};$

and let Mathematica write it all out.

For the objective:

In[*]:= $\mathbf{F} \cdot \mathbf{Y}$

Out[*]:= z

For the inequality constraints:

In[*]:= `Map[# ≥ 0 &, A.Y + B] // TableForm`

Out[*]//TableForm=

$$\begin{array}{l} -2 y2 + 3 y3 + z \geq 0 \\ 2 y1 - 3 y4 + z \geq 0 \\ -3 y1 + 4 y4 + z \geq 0 \\ 3 y2 - 4 y3 + z \geq 0 \\ 1 - y1 \geq 0 \\ 1 - y2 \geq 0 \\ 1 - y3 \geq 0 \\ 1 - y4 \geq 0 \\ y1 \geq 0 \\ y2 \geq 0 \\ y3 \geq 0 \\ y4 \geq 0 \end{array}$$

For the equality constraints :

```
In[ ]:= Map[# == 0 &, A1.Y + B1] // TableForm
Out[ ]//TableForm=
-1 + y1 + y2 + y3 + y4 == 0
```

This is exactly what we had written in Version 1 of the answer directly .

Task 3.2 Write a Monte Carlo simulation for *Morra* that takes a mixed strategy as input and determines the expected utility assuming that both players play this strategy (using a fixed number of iterations given as a third parameter).

Answer 3.2 For Python see separate notebook

```
In[ ]:= averagePayoff[strategy_, payoffMatrix_, rounds_] :=
Mean[Table[Extract[payoffMatrix,
RandomChoice[strategy → Range[Length[strategy]], 2]], {rounds}]]
```

Example of use with P from above :

```
In[ ]:= S = {1 / 5, 2 / 5, 2 / 5, 0};
averagePayoff[S, P, 10 000] // N
Out[ ]:=
0.0137
```

Task 3.3 Using your solution from 1.2 verify whether *Morra* is a fair game.

Answer 3.3: As this is a zero sum game, you know that the payoff for Player B is the negative of the payoff for Player A. Hence you only need to compute one payoff. Compute the payoff for one of the players in Nash. For a fair game this needs to be zero.

```
In[ ]:= S = {0, 4 / 7, 3 / 7, 0};
averagePayoff[S, P, 10 000] // N
Out[ ]:=
0.
```